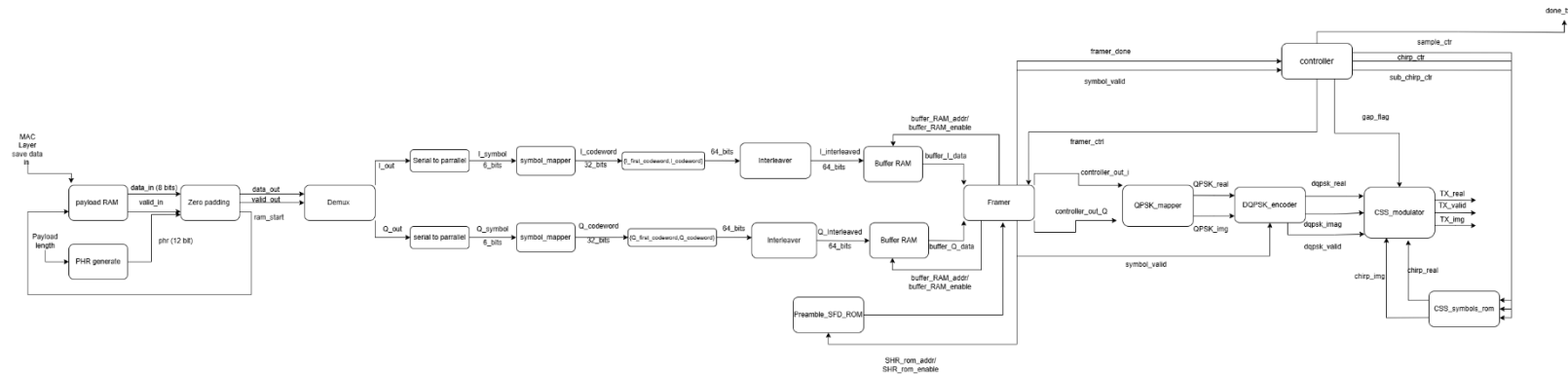


Team 4

Design and FPGA Implementation of an IEEE 802.15.4 Chirp Spread Spectrum (CSS) Physical Layer Transmitter

Block diagram:



System Overview: CSS Transmitter Architecture

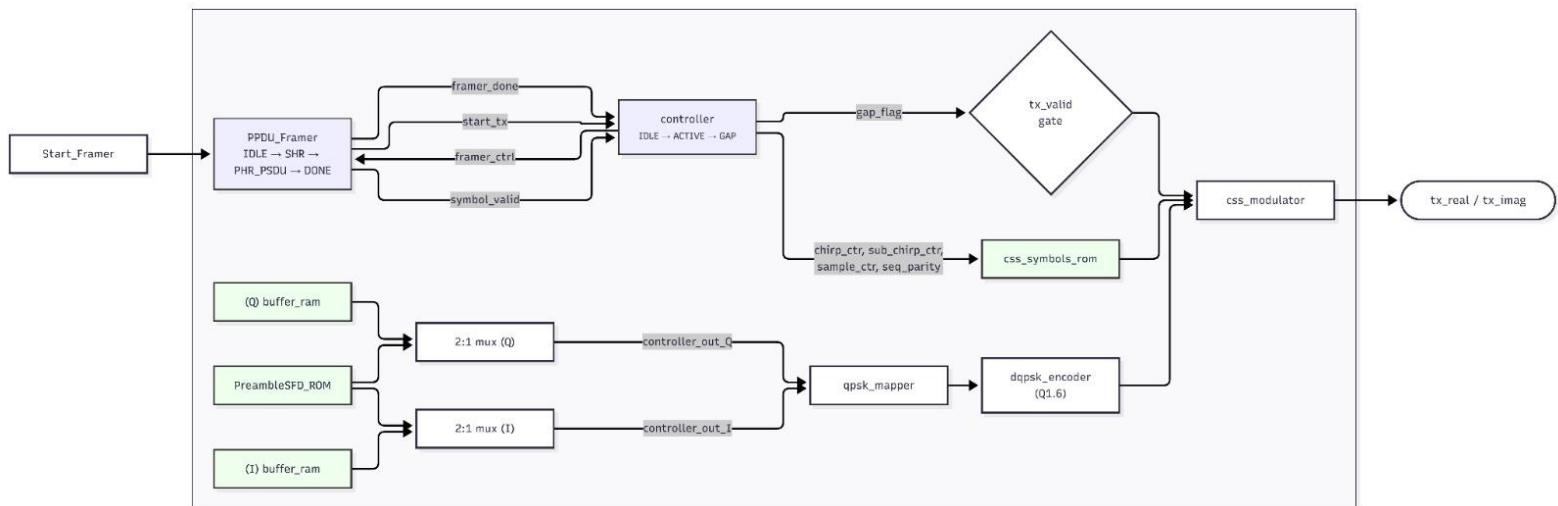
The architecture represents a complete Chirp Spread Spectrum (CSS) transmitter baseband processing chain. The system is divided into digital pre-processing, packet framing, and chirp modulation stages.

1. Data Pre-processing & Interleaving

- **Payload RAM:** Serves as the initial interface between the MAC layer and the PHY layer, storing the raw payload data bytes before transmission begins.
- **Zero Padding:** Appends necessary padding bits to the payload and PHR (PHY Header) to ensure the data stream is a multiple of the required processing block size (e.g., 24 bits for 1 Mbps).
- **Demux:** Splits the incoming serial padded bitstream into two parallel paths: an even stream for the In-phase (I) channel and an odd stream for the Quadrature (Q) channel.

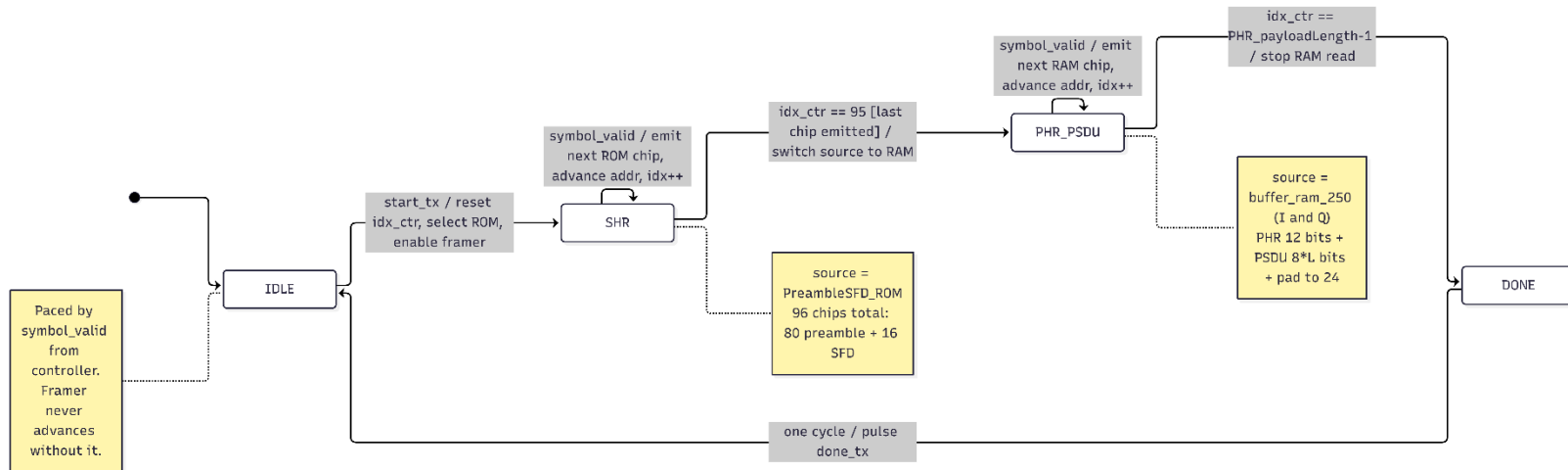
- **Serial to Parallel & Symbol Mapper:** Converts the serial bitstreams into 6-bit symbols and maps them into corresponding 32-bit codewords for both I and Q paths.
- **Interleaver:** Concatenates two 32-bit codewords into a 64-bit block and rearranges the bits to provide robustness against burst errors during transmission.

2. Buffering, Framing & Control



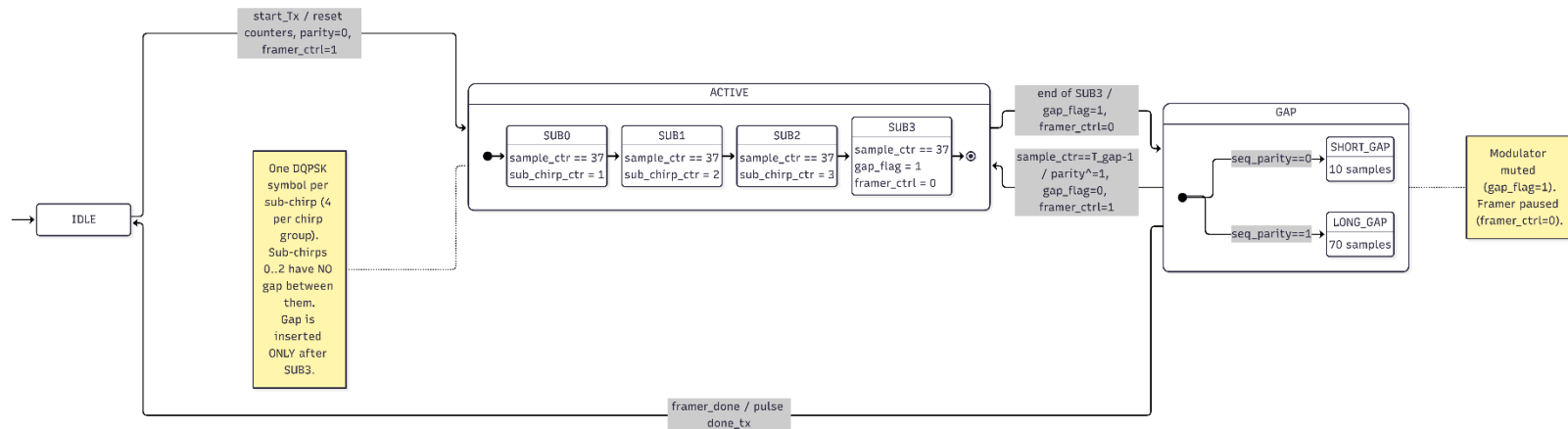
- **Buffer RAM:** Acts as temporary storage for the fully interleaved 64-bit data words for both I and Q channels. It holds the payload data until the Framer signals to read it.
- **Preamble SFD ROM:** A read-only memory block that stores the Synchronization Header (SHR), consisting of the synchronization preamble and the Start-of-Frame Delimiter (SFD).

• Framer:



The core multiplexing unit of the packet builder. It first reads the synchronization header from the Preamble SFD ROM, then smoothly transitions to reading the payload from the Buffer RAMs, outputting a continuous, unified data stream.

• Controller:



The master state machine governing transmission timing. It receives the transmission trigger, enables the Framer via `framer_ctrl`, and generates the synchronization counters (`sample_ctr`, `chirp_ctr`, `sub_chirp_ctr`) required to drive the chirp waveform generation. It also outputs the final `done_Tx` signal when the transmission completes.

3. Modulation & Signal Generation

- **QPSK Mapper:** Receives the framed I and Q chip streams from the Framers and maps them onto a discrete Quadrature Phase Shift Keying (QPSK) constellation.
- **DQPSK Encoder:** Applies differential encoding to the QPSK symbols based on the symbol_valid signal. This resolves phase ambiguities and calculates the required phase transitions for the final waveform.
- **CSS Symbols ROM:** A lookup table driven directly by the Controller's timing counters. It generates the raw, unmodulated baseband chirp waveforms (chirp_real, chirp_imag).
- **CSS Modulator:** The final output stage of the transmitter. It takes the phase-encoded data from the DQPSK Encoder and modulates it onto the baseband chirp waveforms provided by the CSS Symbols ROM, producing the final digital transmission signals (Tx_real and Tx_imag).

Fixed-point word length and format

In digital hardware, binary numbers are represented as either fixed-point or floating-point data types.

Floating-Point Numbers are represented using a sign, an exponent (which dynamically shifts the decimal point), and a mantissa. This allows for an enormous dynamic range, meaning it can represent both incredibly small and incredibly large numbers easily.

Multiplier X Base^{Exponent}

Fixed-Point): The position of the radix point (the decimal point) is fixed. Because there is no dynamic exponent, the hardware treats these numbers as regular integers but assigns a strict, mathematical scaling factor to keep track of fractions

Fixed representation = $\text{round}(\text{floating point} * 2^q)$

benefits of floating-to-fixed-point quantization :

- Reduced Memory Footprint:

Compresses model sizes by 4x or more (e.g., converting 32-bit floats to 8-bit integers), dramatically freeing up storage space.

- Lower Power Consumption: Integer operations require significantly less energy than floating-point math, making it ideal for battery-powered edge devices.
- Accelerated Execution Speed: Integer arithmetic runs much faster on standard hardware, reducing latency and boosting overall system throughput.

1. Chirp ROM:

Since the floating-point source is the **chirp sequence generator**, we use the MSE analysis to determine the required ADC resolution.

The floating-point CSS chirp sequences were quantized using different ROM resolutions. For each resolution, the quantized samples were reconstructed to floating-point values and compared with the original chirp samples. The Mean Squared Error (MSE) was calculated independently for the real and imaginary

components, as well as for the complete complex chirp. The resulting MSE decreases as the number of quantization bits increases, allowing the required ROM resolution to be selected based on the desired accuracy.

And the important equation is:

$$meanSquareError = \left(\frac{\sum_N |exactValue - fixedValue|^2}{\sum_N |exactValue|^2} \right)$$

where:

exactValue: The floating point of chirp sequence samples multiplied by
As R is the analog to digital converter resolution

fixedValue: Round Value of *exactValue*

N= 152: The number of Samples in Chirp Sequence

- **Matlab Code to prove results :**

```
% Parameters
% =====
samplingFreqMhz = 32;
% Test ROM resolutions
R_values = 3:8;
% Number of chirp sequences
num_chirps = 4;
% Number of subchirps
num_subchirps = 4;
% Samples per subchirp
samples_per_subchirp = 38;
%% =====
% Generate chirp sequences
% =====
chirp_data = cell(num_chirps,1);
for m = 1:num_chirps
    chirp_data{m} = chirpSequenceGenerator( ...
        m, samplingFreqMhz);
end
%% =====
% MSE calculation
```

```

% =====
MSE_real = zeros(size(R_values));
MSE_imag = zeros(size(R_values));
MSE_complex = zeros(size(R_values));
for r = 1:length(R_values)
    R = R_values(r);
    % Maximum positive fixed-point value
    scale = 2^(R-1) - 1;
    real_error_all = [];
    imag_error_all = [];
    for m = 1:num_chirps
        seq = chirp_data{m};
        % -----
        % Quantization exactly following the ROM generation method
        % -----
        seq_real_fixed = floor(real(seq) * scale);
        seq_imag_fixed = floor(imag(seq) * scale);
        % -----
        % Reconstruct floating-point value
        % -----
        seq_real_q = seq_real_fixed / scale;
        seq_imag_q = seq_imag_fixed / scale;
        % -----
        % Quantization error
        % -----
        real_error = seq_real_q - real(seq);
        imag_error = seq_imag_q - imag(seq);
        real_error_all = [real_error_all; real_error(:)];
        imag_error_all = [imag_error_all; imag_error(:)];
    end
    % -----
    % MSE
    % -----
    MSE_real(r) = mean(real_error_all.^2);
    MSE_imag(r) = mean(imag_error_all.^2);
    MSE_complex(r) = ...
        mean(real_error_all.^2 + imag_error_all.^2);
end
%% =====
% Display results
% =====
fprintf('\n');
fprintf('===== \n');
fprintf('CHIRP ROM QUANTIZATION MSE SWEEP\n');
fprintf('===== \n');
fprintf('\n');
fprintf('Bits Real MSE Imag MSE Complex MSE\n');

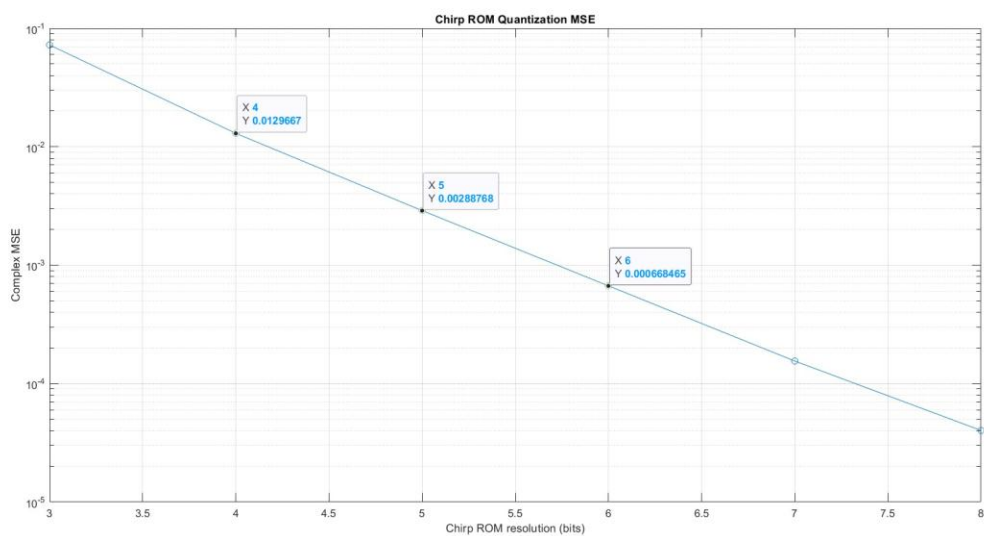
```

```

fprintf('-----\n');
for r = 1:length(R_values)
fprintf('%3d %12.6e %12.6e %6d\n', ...
R_values(r), ...
MSE_real(r), ...
MSE_imag(r), ...
MSE_complex(r));
end
%% =====
% Plot MSE versus ROM resolution
% =====
figure;
semilogy(R_values, MSE_complex, '-o');
grid on;
xlabel('Chirp ROM resolution (bits)');
ylabel('Complex MSE');
title('Chirp ROM Quantization MSE');
%% =====
% Find resolution with minimum MSE
% =====
[min_MSE, index] = min(MSE_complex);
best_R = R_values(index);
fprintf('\n');
fprintf('Minimum complex MSE = %.6e\n', min_MSE);
fprintf('Corresponding resolution = %d bits\n', best_R);

```

- **Results:**



SO

$$S_0 = \cos(\pi/4) + j \sin(\pi/4)$$

and therefore

$$S_0 = 0.70710678 + j0.70710678.$$

After that, because QPSK constellation is

$$\{+1, -1, +j, -j\}$$

the DQPSK operation is only sign changes and I/Q swapping. So **the only real quantization error in the DQPSK block comes from representing 0.70710678.**

- **Matlab code to prove results :**

```
x = cos(pi/4);
F = 6;
scale = 2^F;
% Fixed-point integer representation
x_fixed = round(x * scale);
% Reconstructed floating-point value
x_q = x_fixed / scale;
% Quantization error
q_error = x_q - x;
% MSE
q_MSE = mean(q_error.^2);
fprintf('\n');
fprintf('===== \n');
fprintf('DQPSK Q1.%d QUANTIZATION\n', F);
fprintf('===== \n');
fprintf('Original value = %.12f\n', x);
fprintf('Fixed-point integer = %d\n', x_fixed);
fprintf('Q1.%d value = %.12f\n', F, x_q);
fprintf('Quantization error = %.12f\n', q_error);
fprintf('MSE = %.12e\n', q_MSE);
%% =====
% Check several fractional widths
% =====
F_values = 4:8;
```

```

fprintf('\n');
fprintf('=====\n');
fprintf('Q-FORMAT COMPARISON\n');
fprintf('=====\n');
fprintf(' F Fixed Value Error MSE\n');
fprintf('-----\n');
for F = F_values
scale = 2^F;
x_fixed = round(x * scale);
x_q = x_fixed / scale;
error = x_q - x;
mse = error^2;
fprintf('%2d %4d %.9f %+.9f %.6e\n', ...
F, x_fixed, x_q, error, mse);
end

```

- **Results :**

```

=====
DQPSK Q1.6 QUANTIZATION
=====

```

```

Original value      = 0.707106781187
Fixed-point integer = 45
Q1.6 value          = 0.703125000000
Quantization error  = -0.003981781187
MSE                  = 1.585458141754e-05

```

```

=====
Q-FORMAT COMPARISON
=====

```

F	Fixed	Value	Error	MSE
4	11	0.687500000	-0.019606781	3.844259e-04
5	23	0.718750000	+0.011643219	1.355645e-04
6	45	0.703125000	-0.003981781	1.585458e-05
7	91	0.710937500	+0.003830719	1.467441e-05
8	181	0.707031250	-0.000075531	5.704960e-09

```

..

```

Matlab Verification:

```
>> golden_script
```

```
=====
GOLDEN vs RTL COMPARISON
=====
```

```
[PASS] Real sample count matches (35328)
[PASS] Imag sample count matches (35328)
[PASS] Real file IDENTICAL to golden (35328 samples)
[PASS] Imag file IDENTICAL to golden (35328 samples)
-----
```

```
RESULT: Golden and RTL outputs are IDENTICAL
-----
=====
```

```
% Setup, matching Top.v's actual hardcoded config
dataRate = 1; % 250 kbps -- Top.v ties data_rate=1'b1, uses _250 submodules
simulationParameters
globalSettings()
global DQPSK_output
global Tchirp
global Tsub

chirpIndex = 1;
chirpSequence = chirpSequenceGenerator(chirpIndex, samplingFreqMhz);

% Fixed-point chirp ROM values (already validated). Computed BEFORE the
% ChirpSpreadSpectrum_Tx call now, because we're about to feed this straight
% in as the golden model's chirp input instead of the floating-point one.
chirpSequence_Tx6 = floor(chirpSequence * (2^(TxDACbitNumber-1)-1));

% -----
% Rebuild payload, LSB-first per byte to match payload_ram.v's serialization
% -----
%
fid = fopen('payload.txt','r');
payloadLines = textscan(fid, '%s'); fclose(fid);
payloadLines = payloadLines{1};

payloadBits = [];
for n = 1:length(payloadLines)
```

```

        byteBits = double(payloadLines{n}) - double('0'); % MSB-first as written
        payloadBits = [payloadBits, fliplr(byteBits)];      % -> LSB-first (bit0 first)
    end

% -----
% Golden model: drive ChirpSpreadSpectrum_Tx with the SAME 6-bit chirp ROM
% -----
TxchirpSequences = ChirpSpreadSpectrum_Tx(payloadBits, dataRate, chirpSequence_Tx6);

FRAC = 6;
TxchirpSequences = TxchirpSequences * 2^FRAC;

% ---- Reproduce css_modulator.v's fixed-point arithmetic exactly ----
% MATLAB's .* above already did the full complex multiply, so
% real(TxchirpActive) IS mult_rr-mult_ii and imag(TxchirpActive) IS
% mult_ri+mult_ir -- nothing left to recompute, just shift and truncate.
real_product = real(TxchirpSequences);
imag_product = imag(TxchirpSequences);

tx_r = mod(floor(real_product / 64) + 128, 256) - 128; % >>>6, then 8-bit wrap
tx_i = mod(floor(imag_product / 64) + 128, 256) - 128;

goldenReal = tx_r(:);
goldenImag = tx_i(:);

% Write for the testbench to read
fid_r = fopen('golden_tx_real.mem','w');
fid_i = fopen('golden_tx_imag.mem','w');
for i = 1:length(goldenReal)
    fprintf(fid_r, '%s\n', dec2bin(mod(goldenReal(i),256), 8));
    fprintf(fid_i, '%s\n', dec2bin(mod(goldenImag(i),256), 8));
end
fclose(fid_r);
fclose(fid_i);

% =====
% Load RTL traces and compare against golden
% =====
rtlReal = read_bin_file('.../CSS_PHY_Tx/rtl_tx_real_output.txt');
rtlImag = read_bin_file('.../CSS_PHY_Tx/rtl_tx_imag_output.txt');

nReal = min(length(goldenReal), length(rtlReal));
nImag = min(length(goldenImag), length(rtlImag));

mismatchReal = find(goldenReal(1:nReal) ~= rtlReal(1:nReal), 1, 'first');
mismatchImag = find(goldenImag(1:nImag) ~= rtlImag(1:nImag), 1, 'first');

sameLengthReal = (length(goldenReal) == length(rtlReal));
sameLengthImag = (length(goldenImag) == length(rtlImag));

fprintf('\n');
fprintf('===== \n');
fprintf('GOLDEN vs RTL COMPARISON \n');
fprintf('===== \n');

```

```

% ---- sample counts ----
if sameLengthReal
    fprintf('[PASS] Real sample count matches (%d)\n', length(goldenReal));
else
    fprintf('[FAIL] Real count: golden=%d rtl=%d (diff=%d)\n', ...
        length(goldenReal), length(rtlReal), ...
        length(goldenReal) - length(rtlReal));
end

if sameLengthImag
    fprintf('[PASS] Imag sample count matches (%d)\n', length(goldenImag));
else
    fprintf('[FAIL] Imag count: golden=%d rtl=%d (diff=%d)\n', ...
        length(goldenImag), length(rtlImag), ...
        length(goldenImag) - length(rtlImag));
end

% ---- real channel ----
if sameLengthReal && isempty(mismatchReal)
    fprintf('[PASS] Real file IDENTICAL to golden (%d samples)\n', ...
        length(goldenReal));
elseif ~sameLengthReal
    fprintf('[FAIL] Real file differs (length mismatch)\n');
else
    fprintf('[FAIL] Real first mismatch at sample %d\n', mismatchReal-1);
    fprintf('    golden = %4d (%08s)\n', goldenReal(mismatchReal), ...
        dec2bin(mod(goldenReal(mismatchReal),256),8));
    fprintf('    rtl    = %4d (%08s)\n', rtlReal(mismatchReal), ...
        dec2bin(mod(rtlReal(mismatchReal),256),8));
end

% ---- imag channel ----
if sameLengthImag && isempty(mismatchImag)
    fprintf('[PASS] Imag file IDENTICAL to golden (%d samples)\n', ...
        length(goldenImag));
elseif ~sameLengthImag
    fprintf('[FAIL] Imag file differs (length mismatch)\n');
else
    fprintf('[FAIL] Imag first mismatch at sample %d\n', mismatchImag-1);
    fprintf('    golden = %4d (%08s)\n', goldenImag(mismatchImag), ...
        dec2bin(mod(goldenImag(mismatchImag),256),8));
    fprintf('    rtl    = %4d (%08s)\n', rtlImag(mismatchImag), ...
        dec2bin(mod(rtlImag(mismatchImag),256),8));
end

% ---- overall verdict ----
if sameLengthReal && sameLengthImag && ...
    isempty(mismatchReal) && isempty(mismatchImag)
    fprintf('-----\n');
    fprintf('RESULT: Golden and RTL outputs are IDENTICAL\n');
    fprintf('-----\n');
else
    fprintf('-----\n');
    fprintf('RESULT: Golden and RTL outputs DIFFER\n');
    fprintf('-----\n');
end

```

end

fprintf('=====\n\n');

function v = read_bin_file(path)

fid = fopen(path, 'r');

if fid == -1

error('read_bin_file: cannot open %s', path);

end

C = textscan(fid, '%s');

fclose(fid);

lines = C{1};

n = numel(lines);

v = zeros(n, 1);

for k = 1:n

u = bin2dec(lines{k});

if u >= 128

u = u - 256;

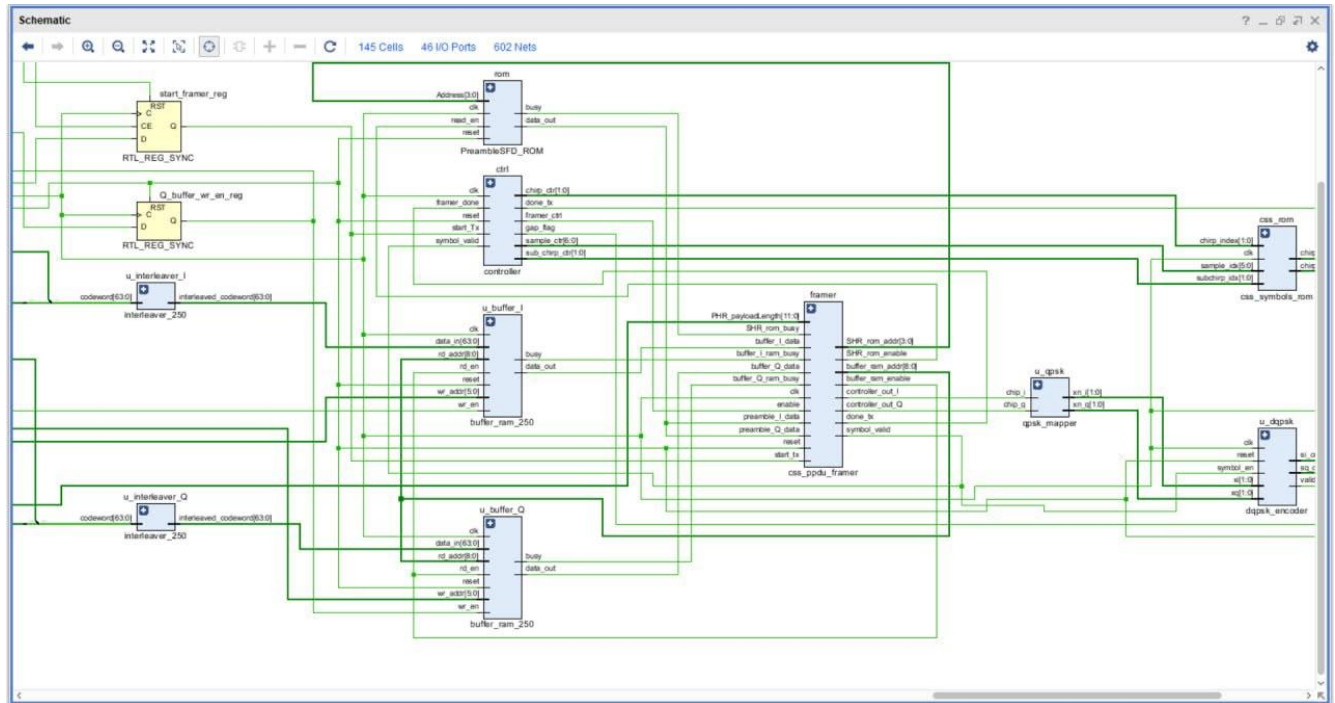
end

v(k) = u;

end

end

FPGA flow:



Intra-Clock Paths - clk - Setup											
Name	Slack	Levels	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination
Path 1	0.483	1	1	modulator/tx_real_reg[4]/C	Tx_real[4]	6.107	3.993	2.115	31.3	clk	clk
Path 2	0.644	2	10	modulator/running_reg/C	Tx_valid	5.932	3.223	2.710	31.3	clk	clk
Path 3	0.868	1	1	modulator/tx_imag_reg[4]/C	Tx_imag[4]	5.719	3.071	2.649	31.3	clk	clk
Path 4	0.883	1	1	modulator/tx_imag_reg[7]/C	Tx_imag[7]	5.703	3.055	2.648	31.3	clk	clk
Path 5	0.897	1	1	modulator/tx_imag_reg[3]/C	Tx_imag[3]	5.690	3.073	2.616	31.3	clk	clk
Path 6	0.907	1	1	modulator/tx_imag_reg[5]/C	Tx_imag[5]	5.680	3.131	2.549	31.3	clk	clk
Path 7	0.914	1	1	modulator/tx_imag_reg[6]/C	Tx_imag[6]	5.673	3.137	2.535	31.3	clk	clk
Path 8	1.021	1	1	ctrl/done_tx_reg/C	done_Tx	5.558	3.115	2.444	31.3	clk	clk
Path 9	1.061	1	1	modulator/tx_real_reg[2]/C	Tx_real[2]	5.513	3.987	1.526	31.3	clk	clk
Path 10	1.061	1	1	modulator/tx_real_reg[5]/C	Tx_real[5]	5.513	3.987	1.526	31.3	clk	clk

Name	Slack	Levels	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source
Path 11	0.104	0	64	I_first_codeword_reg[9]/C	u_buffer_l/ram_reg[0][41]/D	0.444	0.141	0.303	0.0	clk
Path 12	0.110	0	64	I_first_codeword_reg[9]/C	u_buffer_l/ram_reg[9][41]/D	0.446	0.141	0.305	0.0	clk
Path 13	0.117	0	64	I_first_codeword_reg[9]/C	u_buffer_l/ram_reg[1][41]/D	0.429	0.141	0.288	0.0	clk
Path 14	0.136	0	64	I_first_codeword_reg[1]/C	u_buffer_l/ram_reg[56][33]/D	0.469	0.141	0.328	0.0	clk
Path 15	0.144	0	64	I_first_codeword_reg[1]/C	u_buffer_l/ram_reg[20][33]/D	0.460	0.141	0.319	0.0	clk
Path 16	0.146	0	64	I_first_codeword_reg[17]/C	u_buffer_l/ram_reg[5][49]/D	0.471	0.141	0.330	0.0	clk
Path 17	0.149	0	64	I_first_codeword_reg[18]/C	u_buffer_l/ram_reg[41][50]/D	0.482	0.141	0.341	0.0	clk
Path 18	0.152	0	64	I_first_codeword_reg[17]/C	u_buffer_l/ram_reg[25][49]/D	0.491	0.141	0.350	0.0	clk
Path 19	0.168	0	64	I_first_codeword_reg[0]/C	u_buffer_l/ram_reg[14][32]/D	0.502	0.141	0.361	0.0	clk
Path 20	0.170	0	64	Q_first_codeword_reg[0]/C	u_buffer_Q/ram_reg[27][32]/D	0.256	0.141	0.115	0.0	clk

Name	Slack [^] 1	Levels	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock
↳ Path 31	0.081	1	128	payload_din[5]	u_payload_ram/...m_reg[97][5]/D	4.887	0.818	4.069	0.0	wclk
↳ Path 32	0.082	2	128	waddr[2]	u_payload_ram/...reg[20][1]/CE	4.667	0.918	3.749	0.0	wclk
↳ Path 33	0.096	2	128	waddr[2]	u_payload_ram/...reg[106][1]/CE	4.615	0.918	3.697	0.0	wclk
↳ Path 34	0.096	2	128	waddr[2]	u_payload_ram/...reg[106][2]/CE	4.615	0.918	3.697	0.0	wclk
↳ Path 35	0.096	2	128	waddr[2]	u_payload_ram/...reg[106][5]/CE	4.615	0.918	3.697	0.0	wclk
↳ Path 36	0.096	2	128	waddr[2]	u_payload_ram/...reg[106][6]/CE	4.615	0.918	3.697	0.0	wclk
↳ Path 37	0.097	1	128	payload_din[5]	u_payload_ram/...reg[111][5]/D	4.884	0.818	4.066	0.0	wclk
↳ Path 38	0.100	2	128	waddr[2]	u_payload_ram/...reg[120][0]/CE	4.617	0.918	3.699	0.0	wclk
↳ Path 39	0.100	2	128	waddr[2]	u_payload_ram/...reg[120][6]/CE	4.617	0.918	3.699	0.0	wclk
↳ Path 40	0.100	2	128	waddr[2]	u_payload_ram/...reg[120][7]/CE	4.617	0.918	3.699	0.0	wclk

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 0.483 ns	Worst Hold Slack (WHS): 0.081 ns	Worst Pulse Width Slack (WPWS): 9.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 19532	Total Number of Endpoints: 19532	Total Number of Endpoints: 9781

All user specified timing constraints are met.



Tcl Console x Messages Log Reports Design Runs Power DRC Timing ? _ □

INFO: [Vivado 12-1842] Bitgen Completed Successfully.
 INFO: [Project 1-1201] WebTalk data collection is mandatory when using a WebPACK part without a full Vivado license. To see the specific WebTalk data collected for your design
 INFO: [Common 17-83] Releasing license: Implementation
 9 Infos, 3 Warnings, 0 Critical Warnings and 0 Errors encountered.
 write_bitstream completed successfully
 write_bitstream: Time (s): cpu = 00:00:32 ; elapsed = 00:00:29 . Memory (MB): peak = 2610.699 ; gain = 481.500
 output_design.bit

Constrains file:

```
# Defines the system clock running at 32 MHz (31.25 ns period, 50% duty cycle)
create_clock -period 31.250 -name clk [get_ports clk]

# MAC Interface Clock (Example: 50 MHz AXI / System Clock)
create_clock -period 20.000 -name wclk [get_ports wclk]

#####
set_clock_groups -asynchronous -group [get_clocks clk] -group [get_clocks wclk]

# Control Pulse (start_Tx)
set_input_delay -clock [get_clocks clk] -max 19.000 [get_ports start_Tx]
set_input_delay -clock [get_clocks clk] -min 0.450 [get_ports start_Tx]

set_input_delay -clock [get_clocks wclk] -max 16.000 [get_ports {waddr[*]}]
set_input_delay -clock [get_clocks wclk] -min 0.500 [get_ports {waddr[*]}]

set_input_delay -clock [get_clocks wclk] -max 16.000 [get_ports {wen}]
set_input_delay -clock [get_clocks wclk] -min 0.500 [get_ports {wen}]

set_input_delay -clock [get_clocks wclk] -max 16.000 [get_ports {payload_din[*]}]
set_input_delay -clock [get_clocks wclk] -min 0.500 [get_ports {payload_din[*]}]

set_output_delay -clock [get_clocks clk] -max 19.000 [get_ports {Tx_real[*]}]
set_output_delay -clock [get_clocks clk] -min 0.100 [get_ports {Tx_real[*]}]

# Imaginary Part Output Samples (Tx_imag [7:0])
set_output_delay -clock [get_clocks clk] -max 19.000 [get_ports {Tx_imag[*]}]
set_output_delay -clock [get_clocks clk] -min 0.100 [get_ports {Tx_imag[*]}]

# Transmission Complete Status Signal (done_Tx)
set_output_delay -clock [get_clocks clk] -max 19.000 [get_ports done_Tx]
set_output_delay -clock [get_clocks clk] -min 0.100 [get_ports done_Tx]

set_output_delay -clock [get_clocks clk] -max 19.000 [get_ports Tx_valid]
```

```

set_property PACKAGE_PIN F10 [get_ports Reset]

# Start Transmission Pulse -> Up Push Button (BTNU)
set_property PACKAGE_PIN T18 [get_ports start_Tx]

# 8-bit Payload Length / Data Input -> Onboard DIP Switches (SW0 to SW7)
set_property PACKAGE_PIN F22 [get_ports {payloadLength[0]}]
set_property PACKAGE_PIN G22 [get_ports {payloadLength[1]}]
set_property PACKAGE_PIN H22 [get_ports {payloadLength[2]}]
set_property PACKAGE_PIN F21 [get_ports {payloadLength[3]}]
set_property PACKAGE_PIN H19 [get_ports {payloadLength[4]}]
set_property PACKAGE_PIN H18 [get_ports {payloadLength[5]}]
set_property PACKAGE_PIN H17 [get_ports {payloadLength[6]}]
set_property PACKAGE_PIN M15 [get_ports {payloadLength[7]}]

# =====
# 8-bit Real Component Output (Tx_real[7:0])
set_property PACKAGE_PIN T22 [get_ports {Tx_real[0]}]; # "LD0"
set_property PACKAGE_PIN T21 [get_ports {Tx_real[1]}]; # "LD1"
set_property PACKAGE_PIN U22 [get_ports {Tx_real[2]}]; # "LD2"
set_property PACKAGE_PIN U21 [get_ports {Tx_real[3]}]; # "LD3"
set_property PACKAGE_PIN V22 [get_ports {Tx_real[4]}]; # "LD4"
set_property PACKAGE_PIN W22 [get_ports {Tx_real[5]}]; # "LD5"
set_property PACKAGE_PIN U19 [get_ports {Tx_real[6]}]; # "LD6"
set_property PACKAGE_PIN U14 [get_ports {Tx_real[7]}]; # "LD7"

set_property IOSTANDARD LVCMOS33 [get_ports -of_objects [get_iobanks 13]];
set_property IOSTANDARD LVCMOS33 [get_ports -of_objects [get_iobanks 33]];
set_property IOSTANDARD LVCMOS18 [get_ports -of_objects [get_iobanks 34]];
set_property IOSTANDARD LVCMOS18 [get_ports -of_objects [get_iobanks 35]];

set_property SEVERITY {Warning} [get_drc_checks NSTD-1]
set_property SEVERITY {Warning} [get_drc_checks UCIO-1]

```

Tcl Console Messages Log Reports Design Runs x Power DRC Timing ? _ ⌵ ⌴																
Q ⚙ ⚙ ⚙ ⚙ ⚙ ⚙ ⚙ ⚙ ⚙ ⚙ ⚙ ⚙ ⚙ ⚙ ⚙ ⚙ ⚙																
Name	Constraints	Status	WNS	TNS	WHS	THS	TPWS	Total Power	Failed Routes	LUT	FF	BRAMs	URAM	DSP	Start	E
✓ synth_1 (active)	constrs_1	synth_design Complete!								3578	9775	1.00	0	0	9/17/26 11:48 PM	0
✓ impl_1 (active)	constrs_1	route_design Complete!	0.483	0.000	0.081	0.000	0.000	0.127	0	3549	9775	1.00	0	0	9/17/26 11:50 PM	0
✓ synth_2	constrs_1	synth_design Complete!								3578	9775	1.00	0	0	9/18/26 12:10 AM	0
✓ impl_2	constrs_1	route_design Complete!	0.483	0.000	0.081	0.000	0.000	0.127	0	3549	9775	1.00	0	0	9/18/26 12:12 AM	0
✓ synth_4	constrs_1	synth_design Complete!								3573	9778	1.00	0	0	9/18/26 12:10 AM	0
✓ impl_4	constrs_1	route_design Complete!	0.264	0.000	0.028	0.000	0.000	0.127	0	3548	9778	1.00	0	0	9/18/26 12:12 AM	0
✓ synth_6	constrs_1	synth_design Complete!								3981	9704	1.00	0	0	9/18/26 12:10 AM	0
✓ impl_6	constrs_1	route_design Complete!	0.518	0.000	0.095	0.000	0.000	0.127	0	3973	9704	1.00	0	0	9/18/26 12:12 AM	0
✓ synth_9	constrs_1	synth_design Complete!								3587	9722	1.00	0	0	9/18/26 12:10 AM	0
✓ impl_9	constrs_1	route_design Complete!	0.281	0.000	0.063	0.000	0.000	0.125	0	3584	9722	1.00	0	0	9/18/26 12:12 AM	0

Synthesis

Point	Incr	Path

clock clk (rise edge)	0.00	0.00
clock network delay (ideal)	0.00	0.00
Q_wr_addr_reg[1]/CLK (DFFX2)	0.00 #	0.00 r
Q_wr_addr_reg[1]/Q (DFFX2)	0.70	0.70 r
u_buffer_Q/wr_addr[1] (buffer_ram_250_1)	0.00	0.70 r
u_buffer_Q/U10381/ZN (INVX0)	0.57	1.28 f
u_buffer_Q/U10382/ZN (INVX0)	1.70	2.98 r
u_buffer_Q/U7411/Q (AND2X2)	2.64	5.61 r
u_buffer_Q/U4710/QN (AOI21X1)	49.45	55.07 f
u_buffer_Q/U10813/ZN (INVX0)	15.67	70.73 r
u_buffer_Q/U1995/ZN (INVX0)	4.39	75.13 f
u_buffer_Q/U11516/Z (DELLN1X2)	4.06	79.18 fS
u_buffer_Q/U4424/ZN (INVX0)	4.74	83.93 r
u_buffer_Q/U10805/ZN (INVX0)	14.82	98.75 f
u_buffer_Q/U1957/Q (AO22X1)	0.64	99.38 f
u_buffer_Q/ram_reg[35][26]/D (DFFX1)	0.36	99.74 f
data arrival time		99.74
clock clk (rise edge)	39.00	39.00
clock network delay (ideal)	0.00	39.00
clock uncertainty	-0.24	38.76
u_buffer_Q/ram_reg[35][26]/CLK (DFFX1)	0.00	38.76 r
library setup time	-0.18	38.58
data required time		38.58

data required time		38.58
data arrival time		-99.74

slack (VIOLATED)		-61.16

u_buffer_Q/wr_en (buffer_ram_250_1)	0.00	0.68 T
u_buffer_Q/U3020/Q (AND4X4)	3.22	3.90 f
u_buffer_Q/U3121/QN (A0I21X1)	50.54	54.44 r
u_buffer_Q/U2967/Z (DELLN1X2)	2.13	56.57 r
u_buffer_Q/U3035/Z (DELLN1X2)	2.17	58.74 r
u_buffer_Q/U11514/ZN (INVX0)	2.01	60.74 f
u_buffer_Q/U9157/ZN (INVX0)	1.92	62.66 r
u_buffer_Q/U9158/ZN (INVX0)	2.02	64.68 f
u_buffer_Q/U9277/Q (A022X2)	14.87	79.55 f
u_buffer_Q/ram_reg[32][33]/D (DFFX1)	0.36	79.91 f
data arrival time		79.91
clock clk (rise edge)	40.00	40.00
clock network delay (ideal)	0.00	40.00
clock uncertainty	-0.24	39.76
u_buffer_Q/ram_reg[32][33]/CLK (DFFX1)	0.00	39.76 r
library setup time	-0.18	39.58
data required time		39.58
data arrival time		-79.91
slack (VIOLATED)		-40.33

clock clk (rise edge)	0.00	0.00
clock network delay (ideal)	0.00	0.00
framer/buffer_ram_addr_reg[2]/CLK (DFFX1)	0.00 #	0.00 r
framer/buffer_ram_addr_reg[2]/QN (DFFX1)	0.45	0.45 r
framer/U5/ZN (INVX0)	0.14	0.59 f
framer/buffer_ram_addr[2] (css_ppdu_framer_PHR_PSDU_RAM_MEM_DEPTH344_SHR_ROM_MEM_DEPTH12)	0.00	0.59 f
U413/ZN (INVX0)	0.22	0.81 r
U398/ZN (INVX1)	0.16	0.97 f
u_buffer_Q/rd_addr[2] (buffer_ram_250_1)	0.00	0.97 f
u_buffer_Q/U4406/ZN (INVX0)	0.62	1.59 r
u_buffer_Q/U1113/Q (AND2X1)	0.55	2.14 r
u_buffer_Q/U787/Q (AND3X1)	1.60	3.74 r
u_buffer_Q/U4421/ZN (INVX0)	1.70	5.44 f
u_buffer_Q/U4422/ZN (INVX0)	0.46	5.90 r
u_buffer_Q/U15392/Z (DELLN1X2)	2.13	8.03 r
u_buffer_Q/U14406/Z (DELLN2X2)	4.92	12.95 r
u_buffer_Q/U16882/Z (DELLN1X2)	13.84	26.79 r
u_buffer_Q/U16881/QN (AOI221X1)	8.73	28.51 f
u_buffer_Q/U6024/Q (AND4X1)	0.63	29.14 f
u_buffer_Q/U12059/Q (OA22X1)	0.61	29.75 f
u_buffer_Q/U12060/QN (AOI221X1)	0.61	30.36 r
u_buffer_Q/U5/Q (AO222X1)	0.76	31.12 r
u_buffer_Q/shift_reg_reg[61]/D (DFFX1)	0.35	31.48 r
data arrival time		31.48

clock clk (rise edge)	32.00	32.00
clock network delay (ideal)	0.00	32.00
clock uncertainty	-0.24	31.76
u_buffer_Q/shift_reg_reg[61]/CLK (DFFX1)	0.00	31.76 r
library setup time	-0.28	31.48
data required time		31.48

data required time		31.48
data arrival time		-31.48

clock (MET)		0.00

Startpoint: reset (input port clocked by clk)
 Endpoint: u_zero_padding/state_reg[2]
 (rising edge-triggered flip-flop clocked by clk)
 Path Group: clk
 Path Type: min

Des/Clust/Port	Wire Load Model	Library
Top	ForQA	saed90nm_max
zero_padding	8000	saed90nm_max

Point	Incr	Path
clock clk (rise edge)	0.00	0.00
clock network delay (ideal)	0.00	0.00
input external delay	0.00	0.00 r
reset (in)	0.00	0.00 r
u_zero_padding/reset (zero_padding)	0.00	0.00 r
u_zero_padding/U93/QN (NOR3X0)	0.09	0.09 f
u_zero_padding/state_reg[2]/D (DFFX1)	0.04	0.12 f
data arrival time		0.12

clock clk (rise edge)	0.00	0.00
clock network delay (ideal)	0.00	0.00
clock uncertainty	0.16	0.16
u_zero_padding/state_reg[2]/CLK (DFFX1)	0.00	0.16 r
library hold time	-0.04	0.12
data required time		0.12

data required time		0.12
data arrival time		-0.12

slack (MET)		0.01

